

AZURE BLOB STORAGE FILE MANAGEMENT GUIDE

Student Course Management - Assignment Submissions and Course Materials

Storage Account Key -> BlobServiceClient -> Service SAS -> PostgreSQL metadata -> Direct browser upload/download

Scope

This guide deliberately follows the Storage Account Key / Shared Key approach discussed in this conversation. It does not use Microsoft Entra ID as the authentication path. It covers the generic FileAsset design plus assignment submission and teacher workflows.

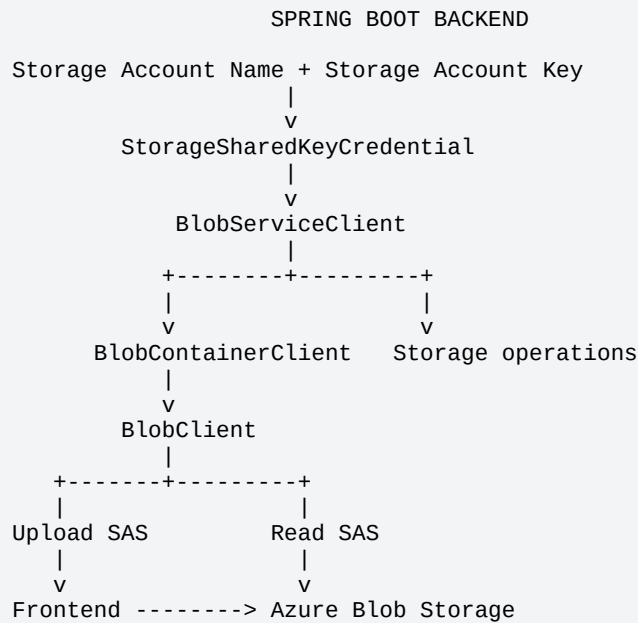
Prepared as a practical reference implementation.

Contents

- 1. Architecture and core mental model
- 2. Azure Storage configuration
- 3. Generic file model: FileAsset
- 4. Assignment submission model
- 5. Upload initiation API
- 6. Frontend direct upload to Azure with SAS
- 7. Complete-submission API and status transition
- 8. View and download with read SAS
- 9. Edit / replace a file
- 10. Delete a file
- 11. Teacher: list all submitted assignments for owned courses
- 12. Teacher: view/download one student submission
- 13. Course materials reuse the same FileAsset layer
- 14. Error handling, consistency, and security rules
- 15. Suggested package structure
- 16. End-to-end request sequences
- 17. Official references

1. Architecture and core mental model

The application separates three concerns: business authorization, PostgreSQL metadata, and Azure Blob bytes. The Storage Account Key never goes to the browser. The browser receives only a narrowly scoped, short-lived SAS when it needs direct access to a specific blob.



PostgreSQL stores metadata and business state; Azure stores file bytes.

1.1 Storage Account Key vs SAS

Credential	Where it lives	Purpose	Typical scope
Storage Account Key	Backend only	Authenticate the backend to the storage account and sign service SAS tokens	Broad storage-account authority
Service SAS	Can be returned to frontend	Delegate limited access to one container/blob for selected permissions and time	Narrow, temporary
Blob URL	Can be stored/referenced	Identifies a blob resource	Does not itself grant private access

Security rule

Never return the Storage Account Key to the frontend. A SAS is a bearer capability: whoever has a valid SAS can use the permissions it grants until it expires or is otherwise invalidated. Generate the smallest permission set needed for the specific operation.

2. Azure Storage configuration

The first layer is the Azure client configuration. The backend reads the account name and account key from configuration, creates **StorageSharedKeyCredential**, and exposes one **BlobServiceClient** bean for reuse.

2.1 Maven dependency

```
<dependency>
  <groupId>com.azure</groupId>
  <artifactId>azure-storage-blob</artifactId>
</dependency>
```

2.2 application.properties

```
azure.storage.account-name=${AZURE_STORAGE_ACCOUNT_NAME}
azure.storage.account-key=${AZURE_STORAGE_ACCOUNT_KEY}
azure.storage.container-name=course-materials
```

Secret handling

Do not commit the real Storage Account Key to Git. Inject it through environment variables or a secret-management system. The code below uses placeholders only.

2.3 AzureStorageConfig

```
package com.studentmanagement.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import com.azure.storage.blob.BlobServiceClient;
import com.azure.storage.blob.BlobServiceClientBuilder;
import com.azure.storage.common.StorageSharedKeyCredential;

@Configuration
public class AzureStorageConfig {

    @Value("${azure.storage.account-name}")
    private String accountName;

    @Value("${azure.storage.account-key}")
    private String accountKey;

    @Bean
    public BlobServiceClient blobServiceClient() {
        StorageSharedKeyCredential credential =
            new StorageSharedKeyCredential(
                accountName,
                accountKey
            );

        return new BlobServiceClientBuilder()
            .endpoint(
                String.format(
                    "https://%s.blob.core.windows.net/",
                    accountName
                )
            )
            .credential(credential)
            .buildClient();
    }
}
```

Mental model: account name identifies the storage endpoint; the account key authenticates the backend; BlobServiceClient is the reusable SDK entry point for Blob Storage.

3. Generic file model: FileAsset

Do not make Submission your generic file table. Submission contains assignment-specific business meaning. FileAsset is the reusable storage record that can be referenced by Submission, CourseMaterial, ExamPaper, TeacherResource, or future file-owning entities.

```
@Entity
@Table(name = "file_assets")
public class FileAsset {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "blob_name", nullable = false, unique = true)
    private String blobName;

    @Column(name = "original_file_name", nullable = false)
    private String originalFileName;

    @Column(name = "container_name", nullable = false)
    private String containerName;

    @Column(name = "content_type")
    private String contentType;

    @Column(name = "file_size")
    private Long fileSize;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private FileStorageStatus status;

    @Column(name = "created_at", nullable = false)
    private LocalDateTime createdAt;

    // getters/setters
}

public enum FileStorageStatus {
    PENDING,
    UPLOADED,
    FAILED
}
```

3.1 Why two status fields?

Entity	Status answers	Example
FileAsset	Did the physical blob reach Azure?	PENDING -> UPLOADED
Submission	Is the assignment submission business operation completed?	PENDING -> SUBMITTED
CourseMaterial	What is the publication state?	DRAFT -> PUBLISHED

Important state rule

Do not set Submission.SUBMITTED merely because a SAS was generated or because the browser says "upload succeeded". The backend should verify the blob before the business state changes.

4. Assignment submission model

```
@Entity
@Table(
```

```

        name = "submissions",
        uniqueConstraints = {
            @UniqueConstraint(
                columnNames = {"student_id", "assignment_id"}
            )
        }
    )
}

public class Submission {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "student_id", nullable = false)
    private Student student;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "assignment_id", nullable = false)
    private Assignment assignment;

    @OneToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "file_asset_id", nullable = false)
    private FileAsset fileAsset;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private SubmissionStatus status;

    @Column(name = "created_at", nullable = false)
    private LocalDateTime createdAt;

    @Column(name = "submitted_at")
    private LocalDateTime submittedAt;

    // getters/setters
}

public enum SubmissionStatus {
    PENDING,
    SUBMITTED,
    FAILED
}

```

4.1 Repositories

```

@Repository
public interface FileAssetRepository
    extends JpaRepository<FileAsset, Long> {
}

@Repository
public interface SubmissionRepository
    extends JpaRepository<Submission, Long>,
        JpaSpecificationExecutor<Submission> {

    Optional<Submission> findByStudentIdAndAssignmentId(
        Long studentId,
        Long assignmentId
    );
}

```

5. Upload initiation API

Because the frontend uploads directly to Azure, the backend endpoint is an initiation endpoint. It does not accept `MultipartFile`. It receives file metadata, performs authorization/business validation, creates `PENDING` metadata rows, generates a blob-specific upload SAS, and returns the SAS URL.

5.1 Request DTO

```
public class SubmissionUploadRequest {

    private String fileName;
    private String contentType;
    private Long fileSize;

    // getters/setters

}
```

5.2 Response DTO

```
public class SubmissionUploadResponse {

    private Long submissionId;
    private Long fileAssetId;
    private String blobName;
    private String uploadUrl;

    // getters/setters

}
```

5.3 AzureBlobStorageService - core helpers and upload SAS

```
@Service
public class AzureBlobStorageService {

    private final BlobServiceClient blobServiceClient;

    @Value("${azure.storage.container-name}")
    private String containerName;

    public AzureBlobStorageService(
        BlobServiceClient blobServiceClient) {
        this.blobServiceClient = blobServiceClient;
    }

    private BlobClient getBlobClient(String blobName) {
        BlobContainerClient containerClient =
            blobServiceClient.getBlobContainerClient(containerName);
        return containerClient.getBlobClient(blobName);
    }

    public String generateUploadSas(String blobName) {
        BlobClient blobClient = getBlobClient(blobName);

        BlobSasPermission permission =
            new BlobSasPermission()
                .setCreatePermission(true)
                .setWritePermission(true);

        OffsetDateTime expiry =
            OffsetDateTime.now().plusMinutes(10);

        BlobServiceSasSignatureValues values =
            new BlobServiceSasSignatureValues(
                expiry,
                permission
            ).setStartTime(
                OffsetDateTime.now().minusMinutes(1)
            );

        String sasToken =
            blobClient.generateSas(values);

        return blobClient.getBlobUrl() + "?" + sasToken;
    }

    public boolean exists(String blobName) {
```

```

        return getBlobClient(blobName).exists();
    }

    public BlobProperties getProperties(String blobName) {
        return getBlobClient(blobName).getProperties();
    }
}

```

5.4 SubmissionService.initiateSubmission

```

@Transactional
public SubmissionUploadResponse initiateSubmission(
    Long assignmentId,
    SubmissionUploadRequest request,
    Authentication authentication) {

    User user = userRepository
        .findByName(authentication.getName())
        .orElseThrow(() ->
            new ResourceNotFoundException("User not found"));

    Student student = studentRepository
        .findById(user.getId())
        .orElseThrow(() ->
            new ResourceNotFoundException("Student not found"));

    Assignment assignment = assignmentRepository
        .findById(assignmentId)
        .orElseThrow(() ->
            new ResourceNotFoundException(
                "Assignment not found"));

    boolean enrolled =
        enrollmentRepository.existsByStudentIdAndCourseId(
            student.getId(),
            assignment.getCourse().getId());

    if (!enrolled) {
        throw new AccessDeniedException(
            "You are not enrolled in this course");
    }

    if (submissionRepository
        .findByStudentIdAndAssignmentId(
            student.getId(), assignmentId)
        .isPresent()) {
        throw new ResourceExistException(
            "Submission already exists");
    }

    String blobName = buildBlobName(
        assignmentId,
        student.getId(),
        request.getFileName());

    FileAsset fileAsset = new FileAsset();
    fileAsset.setBlobName(blobName);
    fileAsset.setOriginalFileName(request.getFileName());
    fileAsset.setContainerName(containerName);
    fileAsset.setContentType(request.getContentType());
    fileAsset.setFileSize(request.getFileSize());
    fileAsset.setStatus(FileStorageStatus.PENDING);
    fileAsset.setCreatedAt(LocalDateTime.now());

    FileAsset savedFileAsset =
        fileAssetRepository.save(fileAsset);

    Submission submission = new Submission();
    submission.setStudent(student);
    submission.setAssignment(assignment);
    submission.setFileAsset(savedFileAsset);
}

```

```

        submission.setStatus(SubmissionStatus.PENDING);
        submission.setCreatedAt(LocalDateTime.now());

        Submission savedSubmission =
            submissionRepository.save(submission);

        String uploadUrl =
            azureBlobStorageService.generateUploadSas(blobName);

        SubmissionUploadResponse response =
            new SubmissionUploadResponse();
        response.setSubmissionId(savedSubmission.getId());
        response.setFileAssetId(savedFileAsset.getId());
        response.setBlobName(blobName);
        response.setUploadUrl(uploadUrl);
        return response;
    }

    private String buildBlobName(
        Long assignmentId,
        Long studentId,
        String originalFileName) {

        String uuid = UUID.randomUUID().toString();

        return "submissions/assignment-"
            + assignmentId
            + "/student-"
            + studentId
            + "/"
            + uuid
            + "-"
            + originalFileName;
    }
}

```

State immediately after initiation

PostgreSQL: FileAsset=PENDING and Submission=PENDING. Azure blob: it may not exist yet. The returned SAS is only an upload capability for the exact blob name generated by the backend.

6. Frontend direct upload to Azure with SAS

The frontend receives the uploadUrl and uses it directly against Azure. The Storage Account Key never leaves the backend. For a simple block-blob PUT, Azure requires the x-ms-blob-type: BlockBlob header.

```

async function uploadToAzure(file, uploadUrl) {

    const response = await fetch(uploadUrl, {
        method: "PUT",
        headers: {
            "x-ms-blob-type": "BlockBlob",
            "Content-Type": file.type
        },
        body: file
    });

    if (!response.ok) {
        throw new Error(`Azure upload failed: ${response.status}`);
    }

    return true;
}

```

Browser CORS

Because the browser calls the Blob endpoint directly, configure Blob service CORS for the real frontend origin

and the required methods/headers. Avoid leaving a fully open production rule.

7. Complete-submission API and status transition

The browser reports that the upload finished, but the backend does not blindly trust that message. The backend loads the Submission, retrieves the blob name from trusted PostgreSQL metadata, asks Azure for blob properties, verifies the blob, then changes the statuses.

7.1 Controller

```
@PostMapping("/submissions/{submissionId}/complete")
public ResponseEntity<Void> completeSubmission(
    @PathVariable Long submissionId,
    Authentication authentication)
    throws AccessDeniedException {

    submissionService.completeSubmission(
        submissionId, authentication);

    return ResponseEntity.ok().build();
}
```

7.2 Azure verification

```
public BlobProperties getProperties(String blobName) {
    return getBlobClient(blobName).getProperties();
}
```

7.3 Service

```
@Transactional
public void completeSubmission(
    Long submissionId,
    Authentication authentication)
    throws AccessDeniedException {

    User user = userRepository
        .findByUsername(authentication.getName())
        .orElseThrow(() ->
            new ResourceNotFoundException("User not found"));

    Student student = studentRepository
        .findById(user.getId())
        .orElseThrow(() ->
            new ResourceNotFoundException("Student not found"));

    Submission submission = submissionRepository
        .findById(submissionId)
        .orElseThrow(() ->
            new ResourceNotFoundException("Submission not found"));

    if (!submission.getStudent().getId()
        .equals(student.getId())) {
        throw new AccessDeniedException(
            "You cannot complete this submission");
    }

    if (submission.getStatus() == SubmissionStatus.SUBMITTED) {
        return; // idempotent completion
    }

    FileAsset fileAsset = submission.getFileAsset();

    BlobProperties properties =
        azureBlobStorageService
            .getProperties(fileAsset.getBlobName());
```

```

    if (fileAsset.getFileSize() != null
        && properties.getBlobSize()
            != fileAsset.getFileSize()) {
        throw new IllegalStateException(
            "Uploaded file size does not match");
    }

    fileAsset.setStatus(FileStorageStatus.UPLOADED);
    submission.setStatus(SubmissionStatus.SUBMITTED);
    submission.setSubmittedAt(LocalDateTime.now());

    fileAssetRepository.save(fileAsset);
    submissionRepository.save(submission);
}

```

7.4 State machine

```

INITIATE
|
+--> FileAsset.PENDING
+--> Submission.PENDING

FRONTEND UPLOAD
|
+--> Azure blob created

COMPLETE
|
+--> backend verifies blob
      |
      +-----+-----+
      |         |         |
      YES       NO       |
      |         |         |
      FileAsset remain
      UPLOADED  PENDING
      Submission
      SUBMITTED

```

Why verify size?

At minimum, verify that the blob exists. If you record file size in PostgreSQL, comparing `BlobProperties.getBlobSize()` with the expected size provides an additional consistency check. You can also verify content type or use stronger integrity metadata if your application needs it.

8. View and download with a read SAS

View and download are both read operations at the Blob Storage layer. Your business API can expose them separately, but the storage permission is the same: read.

```

public String generateReadSas(String blobName) {

    BlobClient blobClient = getBlobClient(blobName);

    BlobSasPermission permission =
        new BlobSasPermission()
            .setReadPermission(true);

    OffsetDateTime expiry =
        OffsetDateTime.now().plusMinutes(10);

    BlobServiceSasSignatureValues values =
        new BlobServiceSasSignatureValues(
            expiry, permission)
            .setStartTime(
                OffsetDateTime.now().minusMinutes(1));
}

```

```

String sasToken =
    blobClient.generateSas(values);

return blobClient.getBlobUrl() + "?" + sasToken;
}

```

8.1 Student view/download

```

@GetMapping("/submissions/{submissionId}/file")
public ResponseEntity<FileAccessResponse> getSubmissionFile(
    @PathVariable Long submissionId,
    Authentication authentication)
    throws AccessDeniedException {

    return ResponseEntity.ok(
        submissionService.getSubmissionFile(
            submissionId, authentication));
}

```

```

@Transactional(readOnly = true)
public FileAccessResponse getSubmissionFile(
    Long submissionId,
    Authentication authentication)
    throws AccessDeniedException {

    // Resolve logged-in student...
    Submission submission = submissionRepository
        .findById(submissionId)
        .orElseThrow(() ->
            new ResourceNotFoundException(
                "Submission not found"));

    // Verify submission belongs to current student.
    // Verify FileAsset.status == UPLOADED.

    FileAsset fileAsset = submission.getFileAsset();

    String readUrl =
        azureBlobStorageService.generateReadSas(
            fileAsset.getBlobName());

    return buildFileAccessResponse(fileAsset, readUrl);
}

```

8.2 View vs download

User action	Azure permission	How frontend behaves
View PDF	READ	Open/use the returned URL or fetch and display content
Download PDF	READ	Fetch the same URL and save the bytes / trigger download

Do not store the SAS URL in PostgreSQL

Store the stable blobName. Generate a fresh read SAS when the user requests access. A SAS URL has an expiry and is not a stable identifier.

9. Edit / replace a file

For an existing blob, an edit/replacement operation can use a write-only SAS. The business service must first decide whether the current user is allowed to edit the resource.

```

public String generateEditSas(String blobName) {
    BlobClient blobClient = getBlobClient(blobName);

    BlobSasPermission permission =
        new BlobSasPermission()
            .setWritePermission(true);

    OffsetDateTime expiry =
        OffsetDateTime.now().plusMinutes(10);

    BlobServiceSasSignatureValues values =
        new BlobServiceSasSignatureValues(
            expiry, permission)
            .setStartTime(
                OffsetDateTime.now().minusMinutes(1));

    String sasToken =
        blobClient.generateSas(values);

    return blobClient.getBlobUrl() + "?" + sasToken;
}

@PutMapping("/submissions/{submissionId}/file")
public ResponseEntity<FileAccessResponse> editSubmissionFile(
    @PathVariable Long submissionId,
    Authentication authentication)
    throws AccessDeniedException {

    // 1. Authorize current student.
    // 2. Verify the submission is still editable.
    // 3. Read the existing FileAsset.blobName.
    // 4. Generate a write SAS for exactly that blob.
    // 5. Return the SAS URL.
    // 6. Frontend uploads the replacement directly to Azure.
    // 7. Complete endpoint verifies the new blob and updates metadata.

    return ResponseEntity.ok(response);
}

```

Design alternative: versioned replacement

For stronger auditability, generate a new unique blobName for each replacement instead of overwriting the old blob. Then update FileAsset.blobName only after the new blob is verified. This is often safer for submissions that need history.

10. Delete a file

For destructive operations, keep deletion backend-controlled rather than giving the browser a delete SAS. The backend authenticates the user, checks business authorization, deletes the blob with its Storage Key-backed BlobClient, and then updates/deletes metadata.

```

public void deleteBlob(String blobName) {
    getBlobClient(blobName).deleteIfExists();
}

@DeleteMapping("/submissions/{submissionId}")
public ResponseEntity<Void> deleteSubmission(
    @PathVariable Long submissionId,
    Authentication authentication)
    throws AccessDeniedException {

    submissionService.deleteSubmission(
        submissionId, authentication);

    return ResponseEntity.noContent().build();
}

```

```

@Transactional
public void deleteSubmission(
    Long submissionId,
    Authentication authentication)
    throws AccessDeniedException {

    Submission submission = submissionRepository
        .findById(submissionId)
        .orElseThrow(() ->
            new ResourceNotFoundException(
                "Submission not found"));

    // 1. Authorize owner/teacher/admin according to business rules.
    // 2. Get FileAsset.blobName.
    // 3. Delete blob using backend Storage Key credentials.
    azureBlobStorageService.deleteBlob(
        submission.getFileAsset().getBlobName());

    // 4. Update or delete DB metadata according to your retention policy.
    submissionRepository.delete(submission);
    fileAssetRepository.delete(submission.getFileAsset());
}

```

Distributed consistency

Azure Blob Storage and PostgreSQL are separate systems; one database transaction cannot atomically commit both. Make delete/complete operations retryable and consider a cleanup/reconciliation job for rare partial failures.

11. Teacher: list all submitted assignments for their courses

The query target should be Submission, because the API is returning submissions. The relationship to authorize/filter is Submission -> Assignment -> Course -> Teacher. You do not need the Enrollment table for the teacher ownership check.

11.1 SubmissionSpecification

```

@Component
public class SubmissionSpecification {

    public Specification<Submission> hasTeacherId(Long teacherId) {

        return (root, query, criteriaBuilder) -> {

            Join<Submission, Assignment> assignmentJoin =
                root.join("assignment");

            Join<Assignment, Course> courseJoin =
                assignmentJoin.join("course");

            Join<Course, Teacher> teacherJoin =
                courseJoin.join("teacher");

            Join<Submission, FileAsset> fileAssetJoin =
                root.join("fileAsset");

            return criteriaBuilder.and(

                criteriaBuilder.equal(
                    teacherJoin.get("id"),
                    teacherId),

                criteriaBuilder.equal(
                    root.get("status"),
                    SubmissionStatus.SUBMITTED),

                criteriaBuilder.equal(

```

```

        fileAssetJoin.get("status"),
        FileStorageStatus.UPLOADED),

        criteriaBuilder.equal(
            courseJoin.get("isDeleted"),
            false)
    );
};
}
}

```

11.2 Repository

```

@Repository
public interface SubmissionRepository
    extends JpaRepository<Submission, Long>,
        JpaSpecificationExecutor<Submission> {

    Optional<Submission> findByStudentIdAndAssignmentId(
        Long studentId, Long assignmentId);
}

```

11.3 Teacher list service

```

@Transactional(readOnly = true)
public TeacherSubmissionsResponse getMyCourseSubmissions(
    Authentication authentication,
    Pageable pageable) {

    User user = userRepository
        .findByName(authentication.getName())
        .orElseThrow(() ->
            new ResourceNotFoundException("User not found"));

    Teacher teacher = teacherRepository
        .findByName(user)
        .orElseThrow(() ->
            new ResourceNotFoundException("Teacher not found"));

    Specification<Submission> specification =
        submissionSpecification.hasTeacherId(
            teacher.getId());

    Page<Submission> page =
        submissionRepository.findAll(
            specification, pageable);

    // Map page content to TeacherSubmissionResponse items.
    // Return pagination fields + student/assignment/file metadata.
    return buildTeacherSubmissionResponse(page);
}

```

11.4 Example response

```

{
  "page": 0,
  "size": 10,
  "totalPages": 3,
  "totalElements": 25,
  "submissions": [
    {
      "submissionId": 501,
      "studentId": 101,
      "studentName": "Student A",
      "assignmentId": 25,
      "assignmentTitle": "Java Assignment",
      "courseId": 15,
      "courseName": "Java",
      "fileName": "java-assignment.pdf",

```

```

        "fileSize": 2457600,
        "submittedAt": "2026-09-18T23:10:00"
    }
  ]
}

```

Do not generate SAS for every row

The list endpoint should return metadata. When the teacher clicks View/Download for one submission, a separate endpoint should authorize that specific submission and generate a short-lived read SAS only for that blob.

12. Teacher: view/download one student submission

12.1 Controller

```

@GetMapping("/teacher/submissions/{submissionId}/file")
public ResponseEntity<FileAccessResponse> getSubmissionFile(
    @PathVariable Long submissionId,
    Authentication authentication)
    throws AccessDeniedException {

    return ResponseEntity.ok(
        submissionService.getTeacherSubmissionFile(
            submissionId, authentication));
}

```

12.2 Authorization + read SAS

```

@Transactional(readOnly = true)
public FileAccessResponse getTeacherSubmissionFile(
    Long submissionId,
    Authentication authentication)
    throws AccessDeniedException {

    User user = userRepository
        .findByUserName(authentication.getName())
        .orElseThrow(() ->
            new ResourceNotFoundException("User not found"));

    Teacher teacher = teacherRepository
        .findByUser(user)
        .orElseThrow(() ->
            new ResourceNotFoundException("Teacher not found"));

    Submission submission = submissionRepository
        .findById(submissionId)
        .orElseThrow(() ->
            new ResourceNotFoundException(
                "Submission not found"));

    Course course = submission.getAssignment().getCourse();

    if (course.getTeacher() == null
        || !course.getTeacher().getId()
            .equals(teacher.getId())) {
        throw new AccessDeniedException(
            "You are not the teacher of this course");
    }

    FileAsset fileAsset = submission.getFileAsset();

    if (fileAsset.getStatus()
        != FileStorageStatus.UPLOADED) {
        throw new ResourceNotFoundException(
            "Submission file is not available");
    }
}

```

```

String readUrl =
    azureBlobStorageService.generateReadSas(
        fileAsset.getBlobName());

return buildFileAccessResponse(fileAsset, readUrl);
}

```

12.3 The authorization chain

```

Teacher request
|
v
Submission
|
+--> Assignment
      |
      +--> Course
            |
            +--> Course.teacher.id

Compare:
Course.teacher.id == loggedInTeacher.id

YES -> generate READ SAS
NO  -> 403 Forbidden

```

13. Course materials reuse the same FileAsset layer

The generic FileAsset design means a future CourseMaterial entity can reuse exactly the same Azure storage service and SAS generation logic.

```

@Entity
@Table(name = "course_materials")
public class CourseMaterial {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "course_id", nullable = false)
    private Course course;

    @OneToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "file_asset_id", nullable = false)
    private FileAsset fileAsset;

    @Column(nullable = false)
    private String title;

    private String description;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private CourseMaterialStatus status;

    @Column(name = "created_at", nullable = false)
    private LocalDateTime createdAt;
}

public enum CourseMaterialStatus {
    DRAFT,
    PUBLISHED,
    ARCHIVED
}

```

Business entities

```

Submission -----+
                  |
CourseMaterial -----+-----> FileAsset -----> Azure Blob
                  |
Future ExamPaper -----+
TeacherResource -----+
  
```

Why not one giant File table?

Avoid a polymorphic table full of nullable assignmentId/courseMaterialId/examId/etc. Keep FileAsset generic and let explicit business entities reference it with foreign keys.

14. Operations, errors, consistency, and security rules

Application action	Storage operation	Recommended client access
View	Read blob	Read-only SAS
Download	Read blob	Read-only SAS
Upload new file	Create + write as required	Short-lived upload SAS
Edit / replace	Write existing blob, or create a new version	Short-lived write SAS
Delete	Delete blob	Prefer backend-controlled deletion

14.1 Error handling

Failure	Expected behavior
Course/assignment/student not found	Return 404 from business layer
Student is not enrolled	Return 403; do not generate SAS
Teacher does not own the course	Return 403; do not generate SAS
Blob missing during complete	Do not move to SUBMITTED
Blob size mismatch	Do not move to SUBMITTED; flag/retry
SAS expired	Frontend requests a new SAS through backend
Azure storage error	Catch/log Azure SDK exception and translate safely

14.2 Important consistency case

Possible distributed failure:

```

Azure upload succeeds
      |
      v
Database update fails
      |
      v
Azure has blob, PostgreSQL still says PENDING
  
```

Recovery:

- keep completion endpoint idempotent
- allow retry
- optionally add a reconciliation/cleanup job

14.3 Security checklist

- Never expose the Storage Account Key to the frontend.
- Generate SAS only after business authorization succeeds.
- Scope SAS to the exact blob or narrowly required container.
- Use the shortest practical expiry time.
- Grant only the permission needed: read, create/write, or write.
- Do not store SAS URLs in PostgreSQL; store stable blobName instead.
- Do not trust the frontend to change business status; verify the blob from the backend.
- For browser-to-Blob uploads, configure Blob CORS for the actual frontend origin.
- Use backend-controlled authorization for delete operations.

15. Suggested package structure

```
com.studentmanagement

config/
  AzureStorageConfig.java

controller/
  SubmissionController.java
  TeacherSubmissionController.java
  CourseMaterialController.java      # future

service/
  AzureBlobStorageService.java
  SubmissionService.java
  CourseMaterialService.java        # future

specification/
  SubmissionSpecification.java

entity/
  FileAsset.java
  FileStorageStatus.java
  Submission.java
  SubmissionStatus.java
  CourseMaterial.java               # future
  CourseMaterialStatus.java         # future

repository/
  FileAssetRepository.java
  SubmissionRepository.java
  CourseMaterialRepository.java     # future

dto/request/
  SubmissionUploadRequest.java

dto/response/
  SubmissionUploadResponse.java
  FileAccessResponse.java
  TeacherSubmissionsResponse.java
```

16. End-to-end request sequences

16.1 Student submission upload

1. Frontend -> POST /assignments/25/submissions/initiate
2. Backend authenticates student
3. Backend verifies enrollment + assignment rules
4. Backend generates unique blobName

5. DB: FileAsset=PENDING
6. DB: Submission=PENDING
7. Backend signs upload SAS with Storage Account Key
8. Backend -> frontend: submissionId + uploadUrl
9. Frontend -> Azure: PUT uploadUrl + file bytes
10. Azure stores blob
11. Frontend -> POST /submissions/{id}/complete
12. Backend -> Azure: getProperties(blobName)
13. Backend verifies blob/size/content type as needed
14. DB: FileAsset=UPLOADED
15. DB: Submission=SUBMITTED

16.2 Student view/download

1. Frontend -> GET /submissions/{id}/file
2. Backend authenticates student
3. Backend verifies submission ownership
4. Backend verifies FileAsset=UPLOADED
5. Backend generates READ SAS for exact blob
6. Backend -> frontend: accessUrl
7. Frontend -> Azure: GET accessUrl
8. Azure validates SAS and returns blob bytes

16.3 Teacher list submissions

1. Teacher -> GET /teacher/submissions?page=0&size=10
2. Backend resolves logged-in Teacher
3. SubmissionSpecification joins Submission -> Assignment -> Course -> Teacher
4. Filter teacher.id = current teacher
5. Filter Submission=SUBMITTED
6. Filter FileAsset=UPLOADED
7. Return paginated metadata only
8. No SAS URL is generated for every row

16.4 Teacher view/download one submission

1. Teacher -> GET /teacher/submissions/501/file
2. Backend loads submission 501
3. Backend resolves submission.assignment.course.teacher
4. Compare with logged-in teacher.id
5. If mismatch -> 403
6. Verify FileAsset=UPLOADED
7. Generate READ SAS for exact blob
8. Return temporary accessUrl
9. Frontend -> Azure: GET accessUrl

17. Official references

The implementation patterns above align with Microsoft Learn documentation for the Azure Blob Storage Java SDK. The conversation deliberately uses the Storage Account Key / service SAS path, while Microsoft currently recommends user delegation SAS when possible.

- **Create a service SAS for a container or blob with Java:** <https://learn.microsoft.com/en-us/azure/storage/blobs/sas-service-create-java>
- **Get started with Azure Blob Storage client library for Java:** <https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blob-java-get-started>
- **Azure Storage Blob client library for Java:** <https://learn.microsoft.com/en-us/java/api/overview/azure/storage-blob-readme?view=azure-java-stable>
- **Put Blob REST API:** <https://learn.microsoft.com/en-us/rest/api/storageservices/put-blob>
- **Manage properties and metadata for a blob with Java:** <https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blob-properties-metadata-java>
- **Create a service SAS / SAS overview:** <https://learn.microsoft.com/en-us/azure/storage/common/storage-sas-overview>